

AI Kurs-Assistent

Entwicklung und Deployment mit Streamlit

Daniel Senften

Version 1.0, 2025-10-17

Inhaltsverzeichnis

1. Lernziele	1
2. Überblick	2
3. Vorbereitung	3
3.1. Benötigte Accounts	3
3.2. API-Schlüssel beschaffen	3
4. Schritt 1: Projekt-Setup	4
4.1. Repository erstellen	4
4.2. Projektstruktur erstellen	4
5. Schritt 2: Abhängigkeiten definieren	5
6. Schritt 3: Kurs-Wissensbasis erstellen	6
7. Schritt 4: Hauptanwendung entwickeln	8
8. Schritt 5: Streamlit-Konfiguration	12
9. Schritt 6: Git-Konfiguration	13
10. Schritt 7: Lokales Testen	14
10.1. Umgebungsvariablen setzen	14
10.2. Funktionstest	14
11. Schritt 8: Deployment auf Streamlit Cloud	15
11.1. Code zu GitHub pushen	15
11.2. Streamlit Cloud Setup	15
11.3. Secrets konfigurieren	15
11.4. App deployen	15
12. Schritt 9: Erfolgskontrolle	17
12.1. □ Checkliste	17
12.2. □ Erfolgsnachweis	17
13. Troubleshooting	18
13.1. Häufige Probleme	18
13.2. Backup-Plan bei technischen Problemen	18
14. Erweiterungsideen	20
14.1. Einfache Erweiterungen	20
14.2. Fortgeschrittene Erweiterungen	20
15. Zusammenfassung	21
16. Hausaufgabe	22

1. Lernziele

Nach dieser Übung können Sie:

- Einen funktionsfähigen AI ChatBot mit Streamlit entwickeln
- OpenAI oder Anthropic APIs in eine Web-Anwendung integrieren
- Eine Streamlit-App auf Streamlit Cloud deployen
- Umgebungsvariablen sicher verwalten
- Grundlegende Fehlerbehandlung implementieren

2. Überblick

In dieser Übung erstellen wir einen **AI Kurs-Assistenten**, der Fragen zum AI Development Kurs beantworten kann. Der Assistent wird:

- Fragen zu Kursinhalten, Übungen und Konzepten beantworten
- Mit einer benutzerfreundlichen Web-Oberfläche ausgestattet sein
- Online verfügbar sein (deployed auf Streamlit Cloud)
- Als praktisches Beispiel für AI-Integration dienen

Zeitraumen: 35 Minuten praktische Umsetzung

3. Vorbereitung

3.1. Benötigte Accounts

Stellen Sie sicher, dass Sie folgende Accounts haben:

1. **GitHub Account** (für Code-Verwaltung)
2. **Streamlit Cloud Account** (kostenlos, mit GitHub verknüpft)
3. **OpenAI Account** ODER **Anthropic Account** (für API-Zugang)

3.2. API-Schlüssel beschaffen

3.2.1. Option A: OpenAI API-Schlüssel

1. Besuchen Sie: <https://platform.openai.com/>
2. Registrieren Sie sich oder melden Sie sich an
3. Navigieren Sie zu "API Keys"
4. Erstellen Sie einen neuen API-Schlüssel
5. **Wichtig:** Kopieren und sicher speichern!

3.2.2. Option B: Anthropic API-Schlüssel

1. Besuchen Sie: <https://console.anthropic.com/>
2. Registrieren Sie sich oder melden Sie sich an
3. Navigieren Sie zu "API Keys"
4. Erstellen Sie einen neuen API-Schlüssel
5. **Wichtig:** Kopieren und sicher speichern!

4. Schritt 1: Projekt-Setup

4.1. Repository erstellen

1. Erstellen Sie ein neues GitHub Repository:
 - Name: `ai-course-assistant`
 - Beschreibung: "AI Assistent für den AI Development Kurs"
 - Public Repository (für Streamlit Cloud)
 - Mit README.md initialisieren
2. Klonen Sie das Repository lokal:

```
git clone https://github.com/IHR-USERNAME/ai-course-assistant.git
cd ai-course-assistant
```

4.2. Projektstruktur erstellen

Erstellen Sie folgende Dateien und Ordner:

```
ai-course-assistant/
├── .streamlit/
│   └── config.toml
├── app.py
├── requirements.txt
├── course_knowledge.py
├── .gitignore
└── README.md
```

5. Schritt 2: Abhängigkeiten definieren

Erstellen Sie die `requirements.txt`:

```
streamlit>=1.28.0  
openai>=1.3.0  
anthropic>=0.7.0  
python-dotenv>=1.0.0
```

6. Schritt 3: Kurs-Wissensbasis erstellen

Erstellen Sie `course_knowledge.py`:

```
"""
Wissensbasis für den AI Development Kurs-Assistenten
"""

COURSE_KNOWLEDGE = """
# AI Development Kurs - Wissensbasis

## Kursübersicht
Der AI Development Kurs umfasst 6 Abende mit je 4 Lektionen (50 Min + 10 Min Pause).

### Abend 1: Grundlagen und No-Code Tools
- Einführung in AI/ML Konzepte
- No-Code/Low-Code AI-Plattformen
- Flowise und Langflow
- Praktische Übungen mit AI-Tools

### Abend 2: Python Setup und Deployment
- Python-Entwicklungsumgebung einrichten
- Git/GitHub Grundlagen
- Streamlit AI-Anwendungen entwickeln
- Erstes Deployment auf Streamlit Cloud
- Einführung in Vektordatenbanken

### Abend 3: LangChain und RAG
- LangChain Framework
- Retrieval Augmented Generation (RAG)
- Vector Stores und Embeddings
- Praktische RAG-Implementierung

### Abend 4: Fortgeschrittene Konzepte
- Erweiterte LangChain Patterns
- Multi-Agent Systeme
- Performance Optimierung

### Abend 5: Eigene Projekte
- Projektplanung und -umsetzung
- Individuelle Betreuung
- Code Reviews

### Abend 6: Präsentationen
- Projektpräsentationen
- Feedback und Diskussion
- Ausblick und Weiterbildung

## Wichtige Konzepte
```


Large Language Models (LLMs)

- GPT-4, Claude, Gemini
- Prompt Engineering
- API-Integration

Vector Stores

- Speicherung von Embeddings
- Similarity Search
- FAISS, ChromaDB, Pinecone

Embeddings

- Text-zu-Vektor Transformation
- Semantische Ähnlichkeit
- OpenAI Embeddings, Sentence Transformers

RAG (Retrieval Augmented Generation)

- Kombination von Retrieval und Generation
- Kontextuelle Antworten
- Wissensbasis-Integration

Tools und Technologien

- Python, Streamlit
- LangChain, OpenAI API
- Git/GitHub
- Flowise, Langflow
- Vector Databases

"""

```
def get_system_prompt():
```

```
    """Erstellt den System-Prompt für den AI Assistenten"""
```

```
    return f"""Du bist ein hilfreicher AI-Assistent für den AI Development Kurs.
```

Deine Aufgabe ist es, Fragen der Kursteilnehmer zu beantworten basierend auf folgender Wissensbasis:

{COURSE_KNOWLEDGE}

Antworte immer:

- Freundlich und hilfsbereit
- Auf Deutsch
- Präzise und verständlich
- Mit praktischen Beispielen wenn möglich
- Ehrlich, wenn du etwas nicht weisst

Wenn eine Frage nicht direkt mit dem Kurs zusammenhängt, verweise höflich auf den Kurskontext zurück.

"""

7. Schritt 4: Hauptanwendung entwickeln

Erstellen Sie `app.py`:

```
import streamlit as st
import os
from course_knowledge import get_system_prompt

# Konfiguration der Streamlit-Seite
st.set_page_config(
    page_title="AI Kurs-Assistent",
    page_icon="🤖",
    layout="wide",
    initial_sidebar_state="expanded"
)

def init_ai_client():
    """Initialisiert den AI-Client basierend auf verfügbaren API-Schlüsseln"""

    # Prüfe OpenAI API-Schlüssel
    openai_key = st.secrets.get("OPENAI_API_KEY") or os.getenv("OPENAI_API_KEY")
    anthropic_key = st.secrets.get("ANTHROPIC_API_KEY") or
os.getenv("ANTHROPIC_API_KEY")

    if openai_key:
        try:
            from openai import OpenAI
            client = OpenAI(api_key=openai_key)
            return client, "openai"
        except ImportError:
            st.error("OpenAI-Bibliothek nicht installiert!")
            return None, None

    elif anthropic_key:
        try:
            import anthropic
            client = anthropic.Anthropic(api_key=anthropic_key)
            return client, "anthropic"
        except ImportError:
            st.error("Anthropic-Bibliothek nicht installiert!")
            return None, None

    else:
        st.error("Kein API-Schlüssel gefunden! Bitte konfigurieren Sie OPENAI_API_KEY
oder ANTHROPIC_API_KEY.")
        return None, None

def get_ai_response(client, client_type, messages):
    """Sendet Anfrage an AI-Service und gibt Antwort zurück"""
```

```

try:
    if client_type == "openai":
        response = client.chat.completions.create(
            model="gpt-3.5-turbo",
            messages=messages,
            max_tokens=500,
            temperature=0.7
        )
        return response.choices[0].message.content

    elif client_type == "anthropic":
        # Anthropic erwartet separaten System-Prompt
        system_message = messages[0]["content"] if messages[0]["role"] == "system"
    else ""

    user_messages = [msg for msg in messages if msg["role"] != "system"]

    response = client.messages.create(
        model="claude-3-haiku-20240307",
        max_tokens=500,
        system=system_message,
        messages=user_messages
    )
    return response.content[0].text

except Exception as e:
    return f"Fehler bei der AI-Anfrage: {str(e)}"

def main():
    """Hauptfunktion der Streamlit-App"""

    # Header
    st.title(" AI Kurs-Assistent")
    st.markdown("*Ihr persönlicher Assistent für den AI Development Kurs*")

    # Sidebar mit Informationen
    with st.sidebar:
        st.header(" Über den Assistenten")
        st.markdown("""
        Dieser AI-Assistent kann Ihnen helfen bei:

        - Kursinhalten und Konzepten
        - Übungen und Aufgaben
        - Tools und Technologien
        - Troubleshooting bei Problemen

        Stellen Sie einfach Ihre Frage!
        """)

        st.header(" Kursübersicht")
        st.markdown("""
        Abend 1: Grundlagen & No-Code

```

```

    **Abend 2:** Python & Deployment
    **Abend 3:** LangChain & RAG
    **Abend 4:** Fortgeschrittene Konzepte
    **Abend 5:** Eigene Projekte
    **Abend 6:** Präsentationen
    """)

# AI-Client initialisieren
client, client_type = init_ai_client()

if not client:
    st.stop()

# Chat-Interface
st.header("👉 Stellen Sie Ihre Frage")

# Session State für Chat-Verlauf
if "messages" not in st.session_state:
    st.session_state.messages = []

# Chat-Verlauf anzeigen
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# Neue Nachricht
if prompt := st.chat_input("Ihre Frage zum AI Development Kurs..."):
    # Benutzernachricht hinzufügen
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    # AI-Antwort generieren
    with st.chat_message("assistant"):
        with st.spinner("Denke nach..."):
            # Nachrichten für AI vorbereiten
            ai_messages = [
                {"role": "system", "content": get_system_prompt()}
            ] + st.session_state.messages

            response = get_ai_response(client, client_type, ai_messages)
            st.markdown(response)

    # AI-Antwort zu Session State hinzufügen
    st.session_state.messages.append({"role": "assistant", "content": response})

# Reset-Button
if st.button("🔄 Chat zurücksetzen"):
    st.session_state.messages = []
    st.rerun()

```

```
if __name__ == "__main__":  
    main()
```

8. Schritt 5: Streamlit-Konfiguration

Erstellen Sie `.streamlit/config.toml`:

```
[theme]
primaryColor = "#FF6B6B"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F0F2F6"
textColor = "#262730"

[server]
headless = true
port = 8501
```

9. Schritt 6: Git-Konfiguration

Erstellen Sie `.gitignore`:

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
env/
venv/
.venv/
pip-log.txt
pip-delete-this-directory.txt

# Streamlit
.streamlit/secrets.toml

# Environment variables
.env
.env.local

# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db
```

10. Schritt 7: Lokales Testen

10.1. Umgebungsvariablen setzen

1. Erstellen Sie eine `.env` Datei (wird nicht committet):

```
# Wählen Sie EINE der beiden Optionen:  
  
# Option A: OpenAI  
OPENAI_API_KEY=ihr-openai-api-schlüssel  
  
# Option B: Anthropic  
ANTHROPIC_API_KEY=ihr-anthropic-api-schlüssel
```

1. Installieren Sie die Abhängigkeiten:

```
pip install -r requirements.txt
```

1. Starten Sie die Anwendung:

```
streamlit run app.py
```

1. Öffnen Sie <http://localhost:8501> in Ihrem Browser

10.2. Funktionstest

Testen Sie folgende Fragen:

- "Was lernen wir in Abend 3?"
- "Wie funktioniert RAG?"
- "Welche Tools verwenden wir im Kurs?"
- "Was ist LangChain?"

11. Schritt 8: Deployment auf Streamlit Cloud

11.1. Code zu GitHub pushen

```
git add .  
git commit -m "Initial AI Course Assistant implementation"  
git push origin main
```

11.2. Streamlit Cloud Setup

1. **Besuchen Sie:** <https://share.streamlit.io/>
2. **Anmelden** mit Ihrem GitHub Account
3. **"New app" klicken**
4. **Repository auswählen:** `ihr-username/ai-course-assistant`
5. **Branch:** `main`
6. **Main file path:** `app.py`
7. **App URL** anpassen (optional)

11.3. Secrets konfigurieren

1. **In der Streamlit Cloud App:** Klicken Sie auf "Settings"
2. **"Secrets" Tab** öffnen
3. **Secrets hinzufügen:**

```
# Wählen Sie EINE der beiden Optionen:  
  
# Option A: OpenAI  
OPENAI_API_KEY = "ihr-openai-api-schlüssel"  
  
# Option B: Anthropic  
ANTHROPIC_API_KEY = "ihr-anthropic-api-schlüssel"
```

1. **"Save" klicken**

11.4. App deployen

1. **"Deploy!" klicken**
2. **Warten** bis Deployment abgeschlossen ist (2-3 Minuten)

3. Ihre App ist live! ☐

12. Schritt 9: Erfolgskontrolle

12.1. ☐ Checkliste

- ☐ App läuft lokal ohne Fehler
- ☐ GitHub Repository ist erstellt und Code gepusht
- ☐ Streamlit Cloud App ist deployed
- ☐ API-Schlüssel sind korrekt konfiguriert
- ☐ Chat-Interface funktioniert
- ☐ AI antwortet auf Kursfragen
- ☐ App ist öffentlich erreichbar

12.2. ☐ Erfolgsnachweis

Teilen Sie Ihre App-URL mit der Klasse!

Format: <https://ihr-app-name.streamlit.app/>

13. Troubleshooting

13.1. Häufige Probleme

13.1.1. Problem: "API-Schlüssel nicht gefunden"

Lösung: 1. Prüfen Sie die Secrets-Konfiguration in Streamlit Cloud 2. Stellen Sie sicher, dass der Schlüssel korrekt kopiert wurde 3. Keine Leerzeichen vor/nach dem Schlüssel

13.1.2. Problem: "Import Error" bei AI-Bibliotheken

Lösung: 1. Prüfen Sie `requirements.txt` auf Tippfehler 2. Stellen Sie sicher, dass alle Pakete aufgelistet sind 3. Redeploy der App nach Änderungen

13.1.3. Problem: App lädt nicht

Lösung: 1. Prüfen Sie die Logs in Streamlit Cloud 2. Syntax-Fehler in `app.py` beheben 3. Stellen Sie sicher, dass `app.py` im Root-Verzeichnis liegt

13.1.4. Problem: AI antwortet nicht sinnvoll

Lösung: 1. Prüfen Sie den System-Prompt in `course_knowledge.py` 2. Erweitern Sie die Wissensbasis bei Bedarf 3. Testen Sie verschiedene Fragestellungen

13.2. Backup-Plan bei technischen Problemen

Falls lokale Installation nicht funktioniert:

1. GitHub Codespaces verwenden:

- Öffnen Sie Ihr Repository auf GitHub
- Klicken Sie auf "Code" → "Codespaces" → "Create codespace"
- Entwickeln Sie direkt im Browser
- **Detaillierte Anleitung:** [docs/exercises/backup-environments-quickstart.adoc](#)

2. Gitpod verwenden:

- Fügen Sie `gitpod.io/#` vor Ihre GitHub Repository URL
- Noch schneller als Codespaces
- **Detaillierte Anleitung:** [docs/backup-environments-guide.adoc](#)

3. Pair Programming:

- Arbeiten Sie mit einem Nachbarn zusammen
- Teilen Sie sich einen funktionierenden Computer

4. Fertige Demo-App:

- Falls alles fehlschlägt: <https://ai-course-demo.streamlit.app/>

- Analysieren Sie den Code auf GitHub

Wichtig: Beide Cloud-Umgebungen sind bereits mit allen notwendigen Paketen vorkonfiguriert!

14. Erweiterungsideen

Nach erfolgreichem Deployment können Sie experimentieren:

14.1. Einfache Erweiterungen

- **Mehr Kursinhalte** in `course_knowledge.py` hinzufügen
- **Styling** mit CSS anpassen
- **Emoji** und **Markdown** für bessere Formatierung

14.2. Fortgeschrittene Erweiterungen

- **Datei-Upload** für Kursunterlagen
- **RAG-Integration** mit Vector Store
- **Multi-Modal** Support (Bilder, PDFs)
- **Feedback-System** für Antwortqualität

15. Zusammenfassung

Herzlichen Glückwunsch! ☑

Sie haben erfolgreich:

☑ Einen AI-ChatBot entwickelt ☑ APIs integriert ☑ Eine Web-App deployed ☑ Cloud-Services konfiguriert ☑ Git/GitHub verwendet

Ihre App ist jetzt live und kann von überall erreicht werden!

16. Hausaufgabe

Bis zum nächsten Kursabend:

1. **Experimentieren** Sie mit Ihrem AI-Assistenten
2. **Erweitern** Sie die Wissensbasis um eigene Fragen
3. **Teilen** Sie die App-URL mit Freunden/Familie
4. **Dokumentieren** Sie interessante Antworten
5. **Bereiten** Sie Fragen für den nächsten Abend vor

Viel Spass beim Experimentieren! □